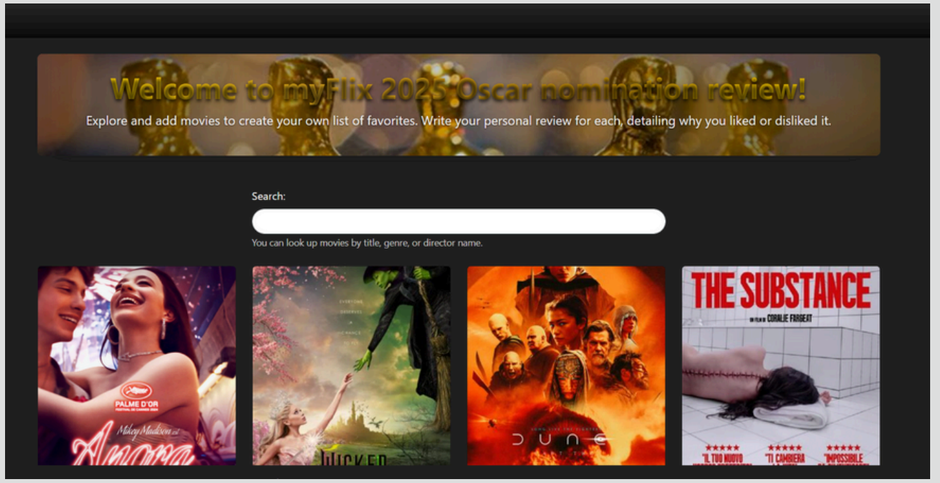
Project Case Study

MYFLIX APP: FULL-STACK MOVIE APPLICATION CASE STUDY

LEVI REDINGER

Project Overview

The myFlix application is a robust, responsive, single-page web application designed for movie enthusiasts. It provides users with a comprehensive platform to explore a collection of Oscar nominated movies, access detailed information, manage their personal profiles, and curate a personalized list of favorite films. This project served as a capstone to demonstrate proficiency in full-stack web development, covering both front-end and back-end development, database management, and user authentication.



- **LIVE DEMO:**
- **FRONTEND (CLIENT-SIDE): HTTPS://OSCARS2025.NETLIFY.APP**
- **BACKEND (API): HTTPS://OSCARS2025-F0070ACEC0C4.HEROKUAPP.COM (BASE URL FOR API ENDPOINTS)**

1. The Problem

Many movie discovery platforms offer limited personalization and often feel cluttered. Users desire a streamlined, intuitive interface where they can easily find information about movies, track their favorites, and manage their profile without unnecessary distractions. Additionally, developers need to demonstrate proficiency in building secure, scalable applications that offer a rich user experience across various devices.

2. Project Goal & Objectives

The primary goal was to build a full-stack movie application that is intuitive, responsive, and secure. Key objectives included:

- User Authentication: Implement secure user registration, login, and logout functionalities.
- Movie Catalog: Display a comprehensive list of movies with detailed information.
- Personalization: Allow users to add/remove movies from a favorites list and manage their personal profile.
- Responsiveness: Ensure the application is fully functional and visually appealing on all screen sizes (mobile, tablet, desktop).
- Robust Backend: Develop a secure RESTful API to manage movie and user data.
- Database Integration: Utilize a NoSQL database (MongoDB) for data storage.

3. My Process & Approach

My development process followed a full-stack agile approach, moving iteratively between front-end and back-end tasks.

Phase 1: Backend Development & Database Design

- **Database Schema:** Designed Mongoose schemas for Movie and User models, including relationships for favorite movies and user-specific comments.
- **RESTful API:** Developed a Node.js and Express.js API, defining endpoints for user registration, login, fetching movies, updating user profiles, and managing favorite movies.
- **Authentication & Authorization:** Implemented JWT-based authentication using Passport.js for secure access to protected routes.
- **Input Validation:** Integrated express-validator to ensure data integrity and security for user inputs.
- **Deployment:** Deployed the backend API to Heroku, ensuring continuous availability.

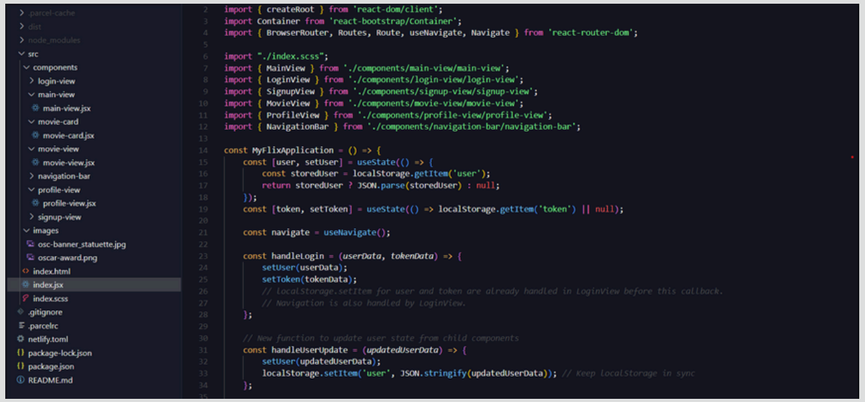
```
// READ a movie by MovieID (UPDATED)
app.get('/movies/:movieId', passport.authenticate('jwt', { session: false }), async (req, res) => {
  try {
    // Check if the provided movieId is a valid MongoDB ObjectId
    if (!mongoose.Types.ObjectId.isValid(req.params.movieId)) {
      return res.status(400).json({ message: 'Invalid Movie ID format.' }); // Corrected to JSON
    }
    const movie = await Movies.findById(req.params.movieId); // Use findById to search by _id
    if (!movie) {
      return res.status(404).json({ message: 'Movie not found.' }); // Corrected to JSON
    }
    res.status(200).json(movie); // Return the movie data
  } catch (err) {
    console.error(err);
    // Specifically catch CastError if mongoose.Types.ObjectId.isValid somehow misses something
    if (err.name === 'CastError' && err.path === '_id') {
      return res.status(400).json({ message: 'Invalid Movie ID format.' }); // Corrected to JSON
    }
    res.status(500).json({ message: 'Error: ' + err.message || 'An unexpected error occurred.' }); // Corrected to JSON
  }
});

// READ a genre by name
app.get('/movies/genres/:name', passport.authenticate('jwt', { session: false }), async (req, res) => {
  await Movies.findOne({ "genre.name": req.params.name })
    .then((movie) => {
      if (movie && movie.genre) { // Check if movie and genre exist
        res.json(movie.genre);
      } else {
        res.status(404).json({ message: 'Genre not found' }); // Corrected to JSON
      }
    })
    .catch((err) => {
      console.error(err);
      res.status(500).json({ message: 'Error: ' + err.message || 'An unexpected error occurred.' }); // Corrected to JSON
```

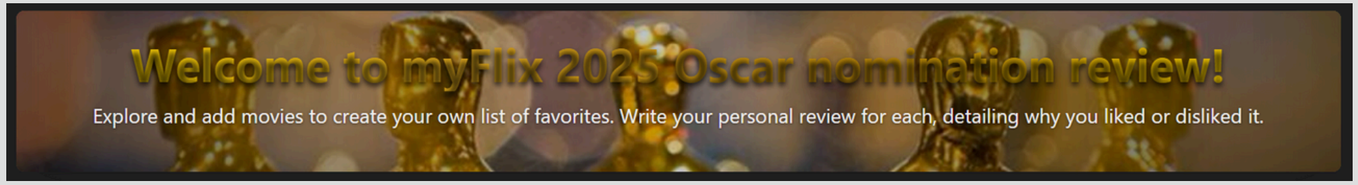
Click the image link to Backend API code and go to index.js

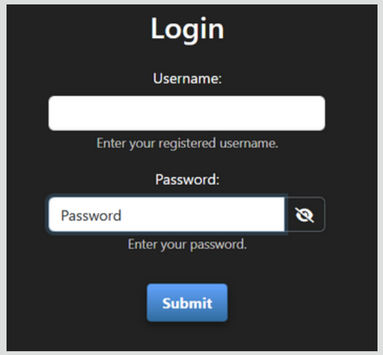
Phase 2: Frontend Development (React)

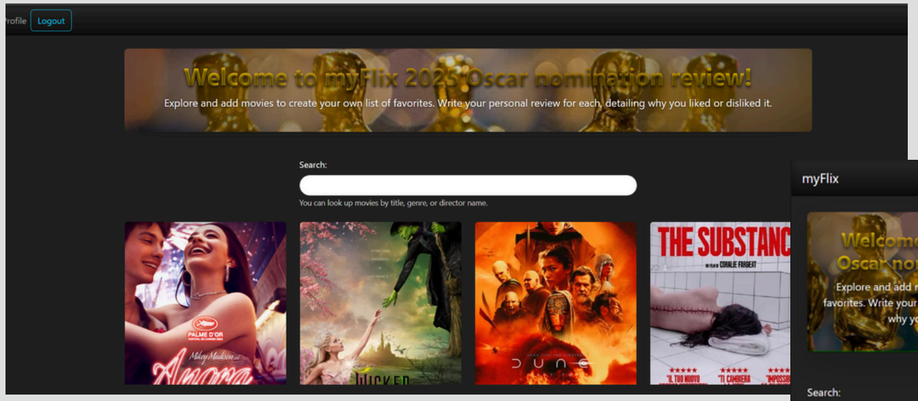
- **Component-Based Architecture:** Built the application using React, breaking down the UI into reusable components (**MainView, MovieCard, MovieView, ProfileView, LoginView, SignupView, NavigationBar**).
- **State Management:** Managed application state using React's useState and useEffect hooks, passing user and token data via props and onUserUpdate callbacks to maintain data consistency across components.
- **Routing:** Utilized React Router DOM for seamless client-side navigation between different views (Home, Movie Details, Profile, Login, Signup).
- **UI Framework:** Leveraged React-Bootstrap for responsive and accessible UI components, minimizing custom CSS for common elements.
- **Styling:** Applied custom SCSS for unique branding elements and enhanced user experience, such as metallic button effects and the welcome graphic.

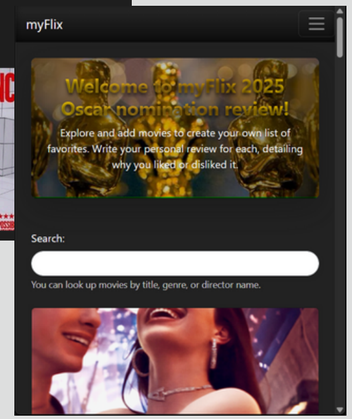
Click the link and navigate through src to index.jsx and other structure folders to see state use and different Views.







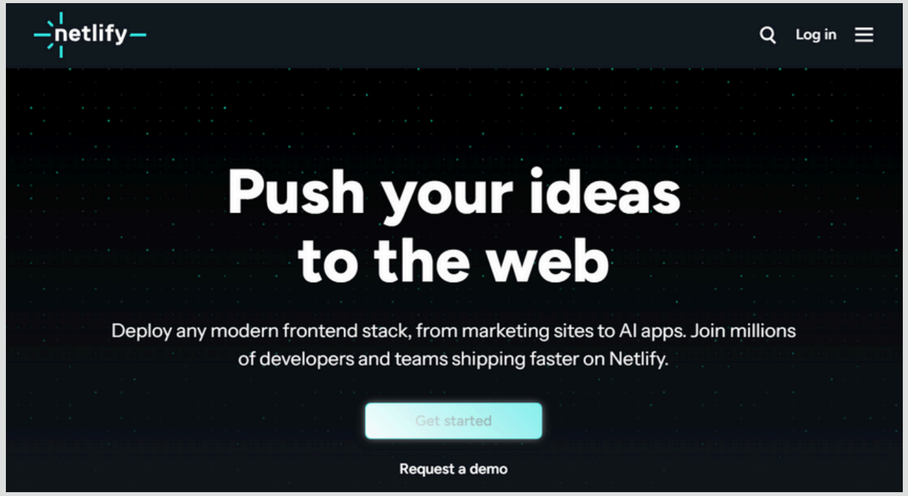


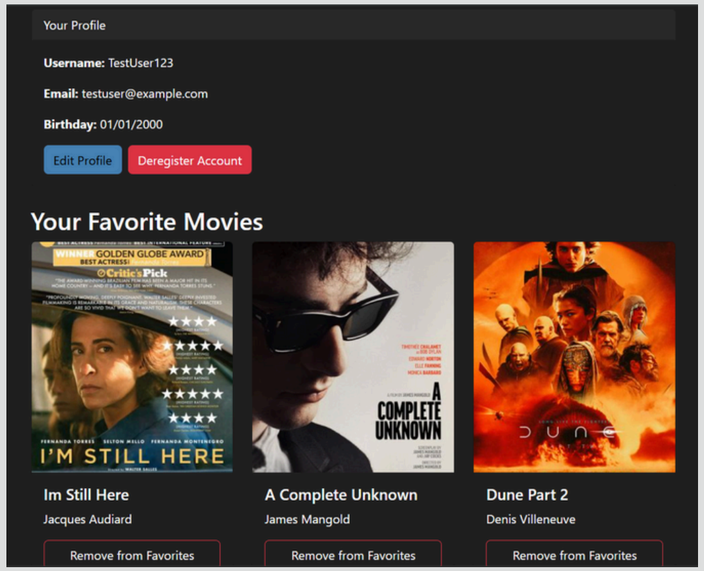


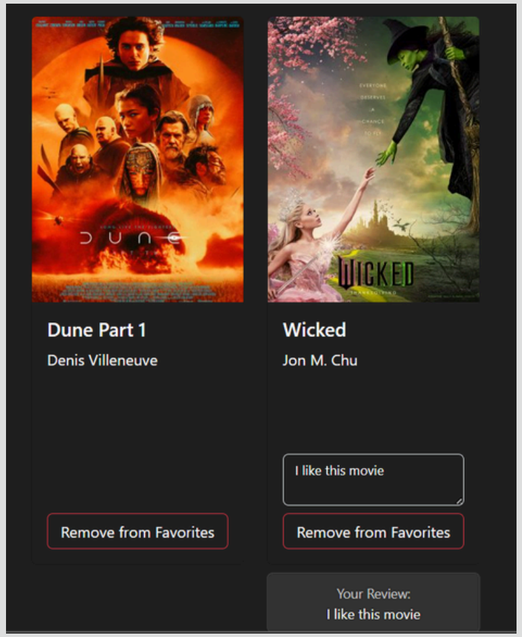
Phase 3: Integration & Refinement

- **API Integration:** Connected the React frontend to the deployed backend API using fetch requests, ensuring data flow and error handling.
- **User Experience (UX) Enhancements:**
  - Implemented a movie search/filter feature allowing users to find movies by title, genre, or director.
  - Added password visibility toggles in login/signup forms for improved usability.
  - Designed a welcome graphic with custom styling and a favicon.
  - Enhanced image sizing for movie posters to ensure they fit correctly without distortion.
- **Responsive Design:** Ensured the application's layout adapts gracefully to various screen sizes, from mobile to desktop, using Bootstrap's grid system.

- **Phase 4: Testing & Deployment**
- **Testing:** While this case study focuses on the build, the project was developed with a TDD mindset, laying the groundwork for comprehensive unit and integration tests (to be explored in future exercises).
- **Deployment:** Deployed the React frontend to Netlify for easy access and sharing.
- **Continuous Improvement:** Addressed bugs and refined features based on testing and observation, particularly focusing on consistent state updates after user actions (e.g., adding/removing favorites).







4. Challenges & Solutions

Developing a full-stack application presented several challenges, each requiring thoughtful solutions:

Challenge 1: Backend Error Response Consistency.

Problem: Early iterations of the backend sometimes sent plain text error messages instead of consistent JSON objects, leading to "Unexpected token" errors on the frontend.

Solution: Thoroughly reviewed all error handling in index.js, ensuring every res.status().send() was converted to res.status().json({ message: ... }) or res.json({ errors: [...] }) to guarantee JSON responses, which the frontend was prepared to parse.

Challenge 2: Inconsistent Favorite Movie Display in Profile.

Problem: After adding or removing a favorite movie, the profile page wouldn't immediately update, or all favorite movies would temporarily disappear. This indicated a state synchronization issue between the MovieCard (where the action occurred) and the ProfileView (where changes needed to reflect).

Solution: Refined the onUserUpdate callback mechanism to ensure the parent MyFlixApplication component reliably updated its user state with the latest data from the backend. Critically, the ProfileView component was refactored to directly depend on the user prop (passed from MyFlixApplication) when calculating and displaying favoriteMoviesDetails. This eliminated an intermediate local state, making the component more reactive to external user data changes. Added defensive checks for movieId and user.favoriteMovies structure to handle populated Mongoose objects and potential null values gracefully.

Challenge 3: Managing Image Paths for Deployment.

Problem: Images appeared correctly in development but broke after deployment due to incorrect relative paths.

Solution: Adjusted image paths in index.html, MovieCard.jsx, MovieView.jsx, and index.scss to use absolute paths (/images/...) or relative paths consistent with the build output (./images/...) for Parcel and Netlify deployment, ensuring assets load correctly in production.

Challenge 4: Parcel Build Failures (Syntax Errors).

Problem: Build process failed due to subtle syntax errors (e.g., => instead of from in an import statement).

Solution: Identified and corrected the precise syntax error in the import statement in MovieCard.jsx, ensuring the code adhered to valid JavaScript syntax for Parcel's transformer.

5. Key Features Implemented

User Management: Secure signup, login, and profile updates.

Personalized Favorites: Users can add and remove movies from a unique list, complete with user-specific comments.

Dynamic Movie Filtering: Efficiently search movies by title, genre, or director.

Responsive Design: Optimal viewing experience across all devices.

Detailed Movie Information: Dedicated views for in-depth movie descriptions.

RESTful API: Robust backend handling data operations and authentication.

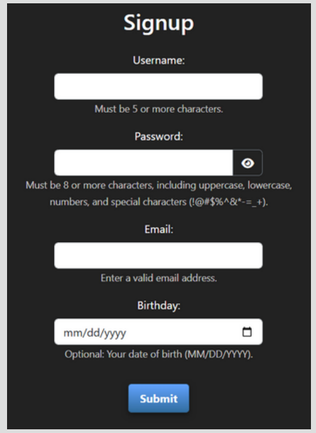
6. Technologies Used

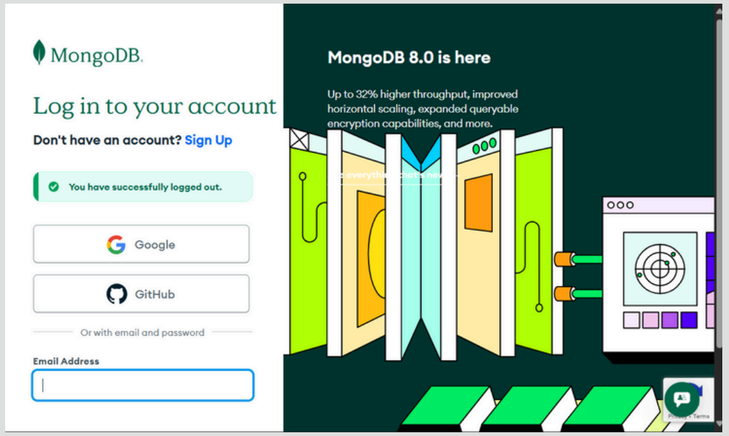
Frontend: React, React-Bootstrap, React Router DOM, SCSS, HTML, JavaScript (ES6+), Parcel

Backend: Node.js, Express.js, MongoDB (Mongoose), bcryptjs, Passport.js, express-validator, CORS

Deployment: Heroku (Backend API), Netlify (Frontend)

Tools: Git, GitHub, VS Code, Postman





7. Outcome & Solution

The myFlix app successfully delivers a functional, user-friendly, and secure movie application. Users can seamlessly browse, search, and manage their favorite movies, enhancing their overall movie discovery experience. The project effectively demonstrates proficiency in building and deploying full-stack web applications, managing state, handling API interactions, and ensuring a responsive design.

8. Lessons Learned & Future Improvements

This project reinforced the importance of meticulous state management in React, robust error handling across both frontend and backend, and careful attention to deployment-specific pathing. It highlighted how seemingly small issues (like a non-JSON error response) can cascade into significant debugging challenges if not addressed systematically.

Future Improvements:

Implement client-side unit and integration tests (using Jest/Enzyme or React Testing Library) to align with TDD principles more deeply.

Enhance accessibility (A11y) features to ensure the application is usable by individuals with disabilities.

Implement data visualization (charts) for movie genre popularity and event distribution (as planned for Project 4.0's "Meet App").

Introduce offline capabilities using service workers to further enhance the PWA experience.

Add a "forgot password" flow.

Allow users to edit existing comments on favorite movies without removing and re-adding.

I invite you to explore the live application and its source code to see the full implementation of these features. Your feedback is highly valued!